

Not only is this the most readable code, it's also the quickest. We know this not because we assumed it, but because we objectively measured it.

The `timeit.timeit()` function can also take a second string argument of SETUP code. The setup code runs only once before running the first string's code. You can also change the default number of trials by passing an integer for the `number` keyword argument. For example, the following test measures how quickly Python's `random` module can generate 10,000,000 random numbers from 1 to 100. (On my machine, it takes about 10 seconds.)

```
>>> timeit.timeit('random.randint(1, 100)', 'import random', number=10000000)
10.020913950999784
```

By default, the code in the string you pass to `timeit.timeit()` won't be able to access the variables and the functions in the rest of the program:

```
>>> import timeit
>>> spam = 'hello' # We define the spam variable.
>>> timeit.timeit('print(spam)', number=1) # We measure printing spam.
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "C:\Users\A\\AppData\Local\Programs\Python\Python37\lib\timeit.py", line
232, in timeit
    return Timer(stmt, setup, timer, globals).timeit(number)
  File "C:\Users\A\\AppData\Local\Programs\Python\Python37\lib\timeit.py", line
176, in timeit
    timing = self.inner(it, self.timer)
  File "<timeit-src>", line 6, in inner
NameError: name 'spam' is not defined
```

To fix this, pass the function the return value of `globals()` for the `globals` keyword argument:

```
>>> timeit.timeit('print(spam)', number=1, globals=globals())
hello
0.000994909999462834
```

A good rule for writing your code is to first make it work and then make it fast. Only once you have a working program should you focus on making it more efficient.